

ISIS-1225

ANÁLISIS DEL RETO

Luis Alejandro Rodríguez Arenas, Cód. 202321287, la.rodriqueza12@uniandes.edu.co

Santiago Rojas, Cod. 202420928, s.rojasg23@uniandes.edu.co

Requerimiento <<1>>

Descripción

Este requerimiento filtra los viajes cuyo pickup_datetime está dentro de un rango dado, los ordena por fecha y hora de recogida ascendente y retorna los N primeros y N últimos (si total $\leq 2N$, retorna todos).

Entrada	Catálogo de viajes, fecha y hora inicial, fecha y hora final, N
Salidas	Diccionario con: tiempo de ejecución, total de viajes, y listas primeros y ultimos con los viajes (cada viaje incluye: fecha/hora de recogida y terminación, coordenadas de recogida y terminación, distancia en millas, costo total pagado).
Implementado (Sí/No)	Si se implementó y lo hizo Luis Alejandro

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tamaño de la lista (lt.size)	$O(1)$
Recorrer todos los viajes (for i in range(size))	$O(n)$
Acceder a cada viaje (lt.get_element)	$O(1)$ por acceso, total $O(n)$
Conversión de fecha (datetime.strptime)	$O(1)$ por viaje, total $O(n)$
Filtro por rango de fechas	$O(1)$ por viaje, total $O(n)$

Ordenamiento (lt.merge_sort)	$O(n \log n)$
Sublistas (lt.sub_list)	$O(1)$
TOTAL	$O(n \log n)$

Análisis

La complejidad total del requerimiento está dominada por el ordenamiento con merge_sort, que es $O(n \log n)$. Todas las demás operaciones son lineales o constantes por viaje. Dado que el tamaño de la lista de trayectos puede ser grande, el tiempo de ejecución crece de forma casi lineal con un ligero factor logarítmico adicional por la ordenación.

Requerimiento <<2>>

Descripción

Este requerimiento identifica los viajes cuya latitud de recogida está en el rango dado, y los ordena de mayor a menor latitud; en empate, de mayor a menor longitud. Retorna los N primeros y N últimos (si total $\leq 2N$, retorna todos).

Entrada	Catálogo de viajes, latitud inicial, latitud final, N
Salidas	Diccionario con: tiempo de ejecución, total de viajes, y listas primeros y ultimos con los viajes (cada viaje incluye: fecha/hora de recogida y terminación, coordenadas de recogida y terminación, distancia en millas, costo total pagado).
Implementado (Sí/No)	Si se implementó y lo hizo Santiago Rojas

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tamaño de la lista (lt.size)	$O(1)$
Recorrer todos los viajes (for i in range(size))	$O(n)$
Acceder a cada viaje (lt.get_element)	$O(1)$ por acceso, total $O(n)$
Conversión de coordenadas a float	$O(1)$ por viaje, total $O(n)$
Filtro por rango de latitud	$O(1)$ por viaje, total $O(n)$

Comparación y ordenamiento (lt.merge_sort)	$O(n \log n)$
TOTAL	$O(n \log n)$

Análisis

El costo principal está en el proceso de ordenamiento por latitud y longitud, con complejidad $O(n \log n)$. Las operaciones de filtrado y acceso son lineales, y no afectan significativamente el crecimiento de la complejidad.

Requerimiento <<4>>

Descripción

Este requerimiento agrupa los viajes por fecha de terminación en una tabla hash, selecciona la fecha solicitada y filtra los viajes que terminan ANTES o DESPUÉS de un tiempo de referencia en esa misma fecha. Los resultados se ordenan del más reciente al más antiguo por dropoff_datetime. Retorna los N primeros y N últimos (si total $\leq 2N$, retorna todos).

Entrada	Catálogo de viajes, fecha de terminación, momento de interés (ANTES o DESPUES), tiempo de referencia, N
Salidas	Diccionario con: tiempo de ejecución, total de viajes, y listas primeros y ultimos con los viajes (cada viaje incluye: fecha/hora de recogida y terminación, coordenadas de recogida y terminación, distancia en millas, costo total pagado).
Implementado (Sí/No)	Si se implementó y se hizo en grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Conversión de fecha y hora (datetime.strptime)	$O(1)$ por viaje, total $O(n)$
Recorrer todos los viajes (for i in range(size))	$O(n)$
Acceder a cada viaje (lt.get_element)	$O(1)$ por acceso, total $O(n)$

Agrupar por fecha (mp.put, mp.get)	$O(1)$ por operación, total $O(n)$
Filtrar por tiempo de referencia	$O(1)$ por viaje, total $O(n)$
Ordenar por fecha de terminación (lt.shell_sort)	$O(n \log n)$
TOTAL	$O(n \log n)$

Análisis

La agrupación en tabla hash mantiene las operaciones de inserción y acceso en $O(1)$ por viaje, pero el ordenamiento de la lista filtrada sigue siendo el factor dominante con $O(n \log n)$. El algoritmo se mantiene eficiente incluso con grandes volúmenes de datos gracias al uso del hash.

Requerimiento <<5>>

Descripción

Este requerimiento agrupa los viajes por **fecha-hora de terminación** en una tabla hash, toma la clave solicitada, ordena los viajes por dropoff_datetime descendente y retorna los **N primeros** y **N últimos** (si total $\leq 2N$, retorna todos).

Entrada	Catálogo de viajes, filtro, fecha inicial, fecha final
Salidas	Diccionario con: tiempo de ejecución, total de viajes, y listas primeros y ultimos con los viajes (cada viaje incluye: fecha/hora de recogida y terminación, coordenadas de recogida y terminación, distancia en millas, costo total pagado).
Implementado (Sí/No)	Si se implementó y se hizo en grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Conversión de fecha y hora (datetime.strptime)	$O(1)$ por viaje, total $O(n)$
Recorrer todos los viajes (for i in range(size))	$O(n)$
Acceder a cada viaje (lt.get_element)	$O(1)$ por acceso, total $O(n)$
Agrupar por fecha (mp.put, mp.get)	$O(1)$ por operación, total $O(n)$

Ordenar por fecha de terminación (lt.shell_sort)	$O(n \log n)$
TOTAL	$O(n \log n)$

Análisis

El uso de la tabla hash permite acceder rápidamente a los trayectos de una fecha-hora específica. El ordenamiento de la lista filtrada vuelve a ser el paso más costoso, con complejidad $O(n \log n)$.

Requerimiento <<6>>

Descripción

Este requerimiento asigna a cada viaje el barrio más cercano del punto de recogida (usando los centroides), agrupa por barrio en una tabla hash, selecciona el barrio solicitado y filtra por rango de horas de recogida. Ordena por pickup_datetime ascendente y retorna los N primeros y N últimos (si total $\leq 2N$, retorna todos).

Entrada	Catálogo de viajes, nombre del barrio, hora inicial, hora final, N
Salidas	Diccionario con: tiempo de ejecución, total de viajes, y listas primeros y ultimos con los viajes (cada viaje incluye: fecha/hora de recogida y terminación, coordenadas de recogida y terminación, distancia en millas, costo total pagado).
Implementado (Sí/No)	Si se hizo en grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Conversión de fecha y hora (datetime.strptime)	$O(1)$ por viaje, total $O(n)$
Recorrer todos los viajes (for i in range(size))	$O(n)$
Acceder a cada viaje (lt.get_element)	$O(1)$ por acceso, total $O(n)$
Determinar barrio más cercano (find_nearest_neighborhood)	$O(n)$
Agrupar por barrio (mp.put, mp.get)	$O(1)$ por operación, total $O(n)$

Filtrar por rango de horas	$O(1)$ por viaje, total $O(n)$
Ordenar por hora de recogida (lt.shell_sort)	$O(n \log n)$
<i>TOTAL</i>	$O(n \log n)$

Análisis

El tiempo está dominado por el ordenamiento de la lista filtrada; el resto de operaciones se mantiene en tiempo lineal o constante por viaje, resultando en $O(n \log n)$.